

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 2 : 머신러닝

DAY6. 머신러닝 차원축소 및 시각화

목 차

1. 머신러닝 차원 축소 이해		3
2. 대표적인 차원 축소 알고리즘		9
3. PCA		22
4. LDA		26
5. t-SNE		30
6. UMAP		33
7. Autoencoder		35

1. 머신러닝 차원 축소 이해

차원 축소(Dimensionality Reduction)

차원 축소(Dimensionality Reduction)는 고차원 데이터(High-Dimensional Data)를 저차원 데이터(Low-Dimensional Data)로 변환하면서 데이터가 가진 중요한 정보를 최대한 유지하는 기법이다. 머신러닝과 딥러닝에서는 데이터의 차원이 너무 높아질 경우 발생하는 차원의 저주(Curse of Dimensionality)를 해결하기 위해 사용한다.

즉, 차원 축소는 데이터가 가진 **변수 또는 특성(feature)의 개수를 줄이는 과정**이다.

예를 들어 데이터에 컬럼이 100개 있다면, 이 100개의 정보를 최대한 유지하면서 2개, 3개, 10개 정도의 적은 변수로 줄이는 방법이다.

원본 데이터: 100차원

↓

차원 축소

↓

축소 데이터: 2차원 또는 10차원

1. 차원의 의미

머신러닝에서 차원은 보통 **입력 변수의 개수**를 의미한다.

예를 들어 학생 데이터를 다음과 같이 가지고 있다고 하자.

이름	키	몸무게	나이	공부시간	출석률
A	170	65	20	5	95

이 데이터에서 입력 변수는 다음 5개이다.

키, 몸무게, 나이, 공부시간, 출석률

따라서 이 데이터는 **5차원 데이터**라고 볼 수 있다.

2. 차원 축소의 필요성

예를 들어 Wine Quality 데이터는 다음과 같은 11개의 특성을 가진다.

fixed acidity

volatile acidity

citric acid
residual sugar
chlorides
free sulfur dioxide
total sulfur dioxide
density
pH
sulphates
alcohol

이를 수학적으로 표현하면

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1,11} \\ x_{21} & x_{22} & \cdots & x_{2,11} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{n,11} \end{bmatrix}$$

이다.

차원이 증가하면

- 학습시간 증가
- 메모리 사용량 증가
- 과적합 증가
- 시각화 불가능

문제가 발생한다.

따라서

11차원

↓

2차원 또는 3차원

으로 축소한다.

1) 계산 속도 향상

변수가 너무 많으면 모델 학습 시간이 길어진다.

특성 1,000개 → 학습 느림

특성 50개 → 학습 빠름

2) 불필요한 변수 제거

모든 변수가 예측에 도움이 되는 것은 아니다.

예를 들어 학생 성적 예측에서 다음 변수는 중요할 수 있다.

공부시간, 출석률, 과제 제출률

하지만 다음 변수는 성적 예측과 관련이 적을 수 있다.

신발 사이즈, 머리카락 길이

이런 불필요한 변수를 줄이면 모델 성능이 좋아질 수 있다.

3) 과적합 감소

변수가 너무 많으면 모델이 학습 데이터의 세세한 특징까지 외워버릴 수 있다.

이를 **과적합(Overfitting)** 이라고 한다.

변수 많음 → 학습 데이터 암기 → 테스트 데이터 성능 저하

차원 축소는 불필요한 정보를 줄여 과적합을 완화할 수 있다.

4) 시각화 가능

사람은 3차원까지만 직관적으로 이해하기 쉽다.

100차원 데이터는 직접 볼 수 없다.

하지만 차원 축소를 통해 2차원 또는 3차원으로 줄이면 그래프로 확인할 수 있다.

100차원 데이터 → 2차원 그래프

3. 차원의 저주

차원이 너무 많아지면 데이터 분석과 머신러닝이 어려워지는 현상을 **차원의 저주(Curse of Dimensionality)** 라고 한다.

차원이 증가하면 다음 문제가 생긴다.

데이터 공간이 급격히 커짐

↓

같은 데이터 개수로는 공간을 충분히 채우지 못함

↓

데이터가 듬성듬성 분포

↓

모델 학습 어려움

예를 들어 1차원에서는 점 10개로 어느 정도 공간을 표현할 수 있다.

1차원: 선 위의 점

하지만 2차원에서는 면적이 생기고, 3차원에서는 부피가 생긴다.

2차원: 평면

3차원: 공간

100차원: 매우 큰 공간

차원이 늘어날수록 필요한 데이터 수가 급격히 증가한다.

4. 차원 축소 방법의 종류

차원 축소는 크게 두 가지 방식으로 나눌 수 있다.

1. 특성 선택(Feature Selection)
2. 특성 추출(Feature Extraction)

5. 특성 선택(Feature Selection)

특성 선택은 기존 변수 중에서 중요한 변수만 골라 사용하는 방법이다.

예를 들어 10개 변수 중 4개만 선택한다.

원본 변수:

x1, x2, x3, x4, x5, x6, x7, x8, x9, x10

선택 후:

x1, x3, x5, x8

기존 변수의 의미가 그대로 유지된다.

특성 선택 방법

1) 필터 방식(Filter Method)

통계적 기준으로 중요한 변수를 선택한다.

예:

상관계수

카이제곱 검정

분산

장점:

빠름

모델과 독립적

단점:

모델 성능과 직접 연결되지 않을 수 있음

2) 래퍼 방식(Wrapper Method)

여러 변수 조합을 직접 모델에 넣어보고 성능이 좋은 조합을 찾는다.

예:

전진 선택법

후진 제거법

RFE

장점:

모델 성능 기준으로 선택

단점:

계산 비용이 큼

3) 임베디드 방식(Embedded Method)

모델 학습 과정에서 중요한 변수를 자동으로 선택한다.

예:

Lasso

Decision Tree

Random Forest

XGBoost

장점:

학습과 변수 선택을 동시에 수행

단점:

모델 종류에 영향을 받음

6. 특성 추출(Feature Extraction)

특성 추출은 기존 변수들을 조합하여 새로운 변수를 만드는 방식이다.

예를 들어 원래 변수 5개를 조합해서 새로운 변수 2개를 만든다.

원본 변수:

x_1, x_2, x_3, x_4, x_5

변환 후:

z_1, z_2

여기서 z_1, z_2 는 원래 변수들의 조합이다.

기존 변수의 의미는 약해질 수 있지만, 데이터의 핵심 구조를 잘 압축할 수 있다.

차원 축소 알고리즘 분류

차원 축소

├── 선형 차원 축소

| ├── PCA

| └── LDA

|

└── 비선형 차원 축소

 ├── t-SNE

 ├── UMAP

 └── Autoencoder

2. 대표적인 차원 축소 알고리즘

차원 축소 알고리즘

차원 축소 알고리즘은 많은 입력 변수(feature)를 적은 수의 변수로 줄이는 방법이다. 목적은 **계산량 감소, 노이즈 제거, 과적합 완화, 시각화, 모델 성능 개선**이다.

대표 알고리즘은 다음과 같다.

1. PCA

PCA는 비지도학습 기반 차원 축소이며, **Principal Component Analysis**, 즉 **주성분 분석**이다. 고차원 데이터에서 분산이 가장 큰 방향을 찾아 새로운 축을 만든다. 이 새로운 축을 **주성분 (Principal Component)** 이라고 한다.

예를 들어 원본 변수가 10개라면 PCA는 이 변수들을 조합하여 새로운 변수 PC1, PC2, PC3 등을 만든다.

원본 변수: $x_1, x_2, x_3, \dots, x_{10}$

↓

PCA 변환

↓

새 변수: PC1, PC2, PC3

PCA의 핵심 목적은 **데이터의 정보 손실을 최소화하면서 차원을 줄이는 것**이다.

PCA 동작 과정

1. 데이터 표준화
2. 공분산 행렬 계산
3. 고유값과 고유벡터 계산
4. 분산이 큰 주성분 선택
5. 선택된 주성분으로 데이터 변환

PCA의 특징

항목	설명
학습 방식	비지도학습
목적	분산이 큰 방향 찾기
장점	빠르고 해석이 비교적 쉬움
단점	선형 구조만 잘 표현
주 사용 목적	차원 축소, 시각화, 노이즈 제거

PCA 사용 예

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)
```

2. LDA

LDA는 **Linear Discriminant Analysis**, 즉 **선형 판별 분석**이다.

PCA와 비슷하게 차원을 줄이지만, LDA는 **정답 라벨 y**를 사용한다.

PCA는 분산이 큰 방향을 찾지만, LDA는 **클래스를 가장 잘 구분하는 방향**을 찾는다.

PCA: 데이터가 넓게 퍼지는 방향

LDA: 클래스가 잘 나뉘는 방향

LDA의 목적

LDA는 다음 두 가지를 동시에 만족하는 축을 찾는다.

1. 같은 클래스끼리는 가깝게
2. 다른 클래스끼리는 멀게

예를 들어 신용등급 데이터를 분류할 때, 좋은 고객과 나쁜 고객이 잘 분리되는 방향을 찾는다.

LDA의 특징

항목	설명
학습 방식	지도학습
필요 데이터	X와 y 모두 필요
목적	클래스 분리
장점	분류 문제에 효과적
단점	선형 분리 가정
주 사용 목적	분류 전처리, 시각화

LDA 사용 예

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda = LinearDiscriminantAnalysis(n_components=1)
X_lda = lda.fit_transform(X, y)
```

3. t-SNE

t-SNE는 **t-distributed Stochastic Neighbor Embedding**의 약자이다.

고차원 데이터의 가까운 데이터끼리의 관계를 유지하면서 2차원 또는 3차원으로 줄이는 알고리즘이다.

주로 **데이터 시각화**에 사용된다.

```
100차원 데이터
↓
t-SNE
↓
2차원 그래프
```

t-SNE의 핵심 개념

t-SNE는 고차원 공간에서 가까운 데이터들이 저차원 공간에서도 가깝게 배치되도록 한다.

예를 들어 이미지 데이터에서 비슷한 숫자 이미지끼리 모이도록 표현할 수 있다.

숫자 0 이미지들 → 한 군집

숫자 1 이미지들 → 다른 군집

숫자 2 이미지들 → 다른 군집

t-SNE의 특징

항목	설명
학습 방식	비지도 학습
목적	시각화
장점	복잡한 비선형 구조 표현 가능
단점	느림, 결과 재현성 낮음
주 사용 목적	고차원 데이터 군집 시각화

t-SNE 사용 예

```
from sklearn.manifold import TSNE
```

```
tsne = TSNE(  
    n_components=2,  
    perplexity=30,  
    learning_rate=200,  
    random_state=42  
)
```

```
X_tsne = tsne.fit_transform(X)
```

4. UMAP

UMAP은 **Uniform Manifold Approximation and Projection**의 약자이다.

t-SNE와 비슷하게 고차원 데이터를 2차원 또는 3차원으로 줄여 시각화하는 알고리즘이다.

하지만 일반적으로 t-SNE보다 빠르고, 큰 데이터에도 비교적 잘 동작한다.

UMAP의 특징

항목	설명
학습 방식	비지도학습
목적	시각화, 차원 축소
장점	빠름, 대용량 데이터에 유리
단점	파라미터에 따라 결과 차이 큼
주 사용 목적	군집 시각화, 임베딩 축소

UMAP 사용 예

```
import umap

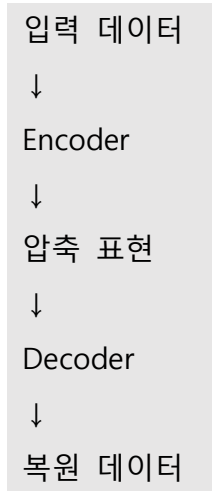
umap_model = umap.UMAP(
    n_components=2,
    n_neighbors=15,
    min_dist=0.1,
    random_state=42
)

X_umap = umap_model.fit_transform(X)
```

5. Autoencoder

Autoencoder는 딥러닝 기반 차원 축소 알고리즘이다.

입력 데이터를 압축한 뒤 다시 복원하도록 학습하는 신경망이다.

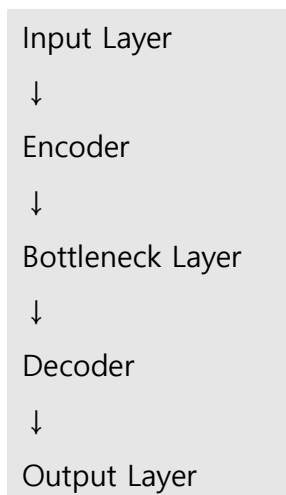


중간의 압축 표현을 **잠재 벡터(latent vector)** 라고 한다.

예를 들어 784차원 MNIST 이미지를 32차원으로 압축할 수 있다.

784차원 이미지 → 32차원 잠재 벡터

Autoencoder 구조



Bottleneck Layer가 원본보다 작은 차원을 가지므로 차원 축소 효과가 발생한다.

Autoencoder의 특징

항목	설명
학습 방식	비지도학습
기본 기술	딥러닝
목적	압축, 복원, 노이즈 제거
장점	비선형 구조 학습 가능
단점	학습 시간이 오래 걸림
주 사용 목적	이미지 압축, 이상탐지, 표현학습

Autoencoder PyTorch 예

```
import torch
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 32)
        )

        self.decoder = nn.Sequential(
            nn.Linear(32, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        out = self.decoder(z)
        return out
```

6. SVD

SVD는 **Singular Value Decomposition**, 즉 **특이값 분해**이다.

행렬을 세 개의 행렬로 분해하여 중요한 정보만 남기는 방법이다.

주로 텍스트 데이터, 추천 시스템, 이미지 압축 등에 사용된다.

원본 행렬 A

↓

U, Σ, V^T 로 분해

수식은 다음과 같다.

$$A = U\Sigma V^T$$

여기서 큰 특이값만 사용하면 원본 정보를 압축할 수 있다.

SVD의 특징

항목	설명
학습 방식	비지도학습
목적	행렬 압축
장점	희소 행렬에 유리
단점	비선형 구조 표현 어려움
주 사용 목적	추천 시스템, 텍스트 분석, 이미지 압축

SVD 사용 예

```
from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(n_components=100)
X_svd = svd.fit_transform(X)
```

7. ICA

ICA는 **Independent Component Analysis**, 즉 **독립 성분 분석**이다.

데이터를 서로 독립적인 성분으로 분리하는 알고리즘이다.

대표적인 예는 **칵테일 파티 문제**이다.

여러 사람이 동시에 말하는 소리가 섞여 있을 때, 각각의 사람 목소리를 분리하는 문제이다.

섞인 신호

↓

ICA

↓

독립된 신호들

ICA의 특징

항목	설명
학습 방식	비지도학습
목적	독립 성분 추출
장점	섞인 신호 분리에 유리
단점	일반 데이터 해석 어려움
주 사용 목적	음성 분리, 신호 처리, 뇌파 분석

ICA 사용 예

```
from sklearn.decomposition import FastICA

ica = FastICA(n_components=3, random_state=42)
X_ica = ica.fit_transform(X)
```

8. NMF

NMF는 **Non-negative Matrix Factorization**, 즉 **비음수 행렬 분해**이다.

데이터 값이 0 이상인 경우에 사용하는 행렬 분해 방법이다.

예를 들어 문서-단어 행렬에서 주제를 찾거나, 이미지에서 부분 패턴을 찾는 데 사용된다.

NMF의 핵심

원본 행렬을 두 개의 작은 행렬로 분해한다.

$$V \approx W \times H$$

단, 모든 값은 0 이상이어야 한다.

NMF의 특징

항목	설명
학습 방식	비지도학습
조건	데이터가 음수가 아니어야 함
장점	결과 해석이 비교적 쉬움
단점	음수 데이터에는 부적합
주 사용 목적	토픽 모델링, 이미지 분석

NMF 사용 예

```
from sklearn.decomposition import NMF

nmf = NMF(n_components=5, random_state=42)
X_nmf = nmf.fit_transform(X)
```

9. Isomap

Isomap은 비선형 차원 축소 알고리즘이다.

데이터가 곡면 위에 분포한다고 보고, 이 곡면 구조를 보존하면서 차원을 줄인다.

예를 들어 말려 있는 종이처럼 생긴 3차원 데이터를 2차원으로 펼치는 데 사용할 수 있다.

Isomap의 특징

항목	설명
학습 방식	비지도학습
목적	비선형 구조 보존
장점	곡면 구조 표현 가능
단점	계산 비용 큼
주 사용 목적	비선형 데이터 시각화

Isomap 사용 예

```
from sklearn.manifold import Isomap

isomap = Isomap(
    n_components=2,
    n_neighbors=5
)

X_isomap = isomap.fit_transform(X)
```

10. 알고리즘별 비교

알고리즘	지도/비지도	선형/비선형	주요 목적	특징
PCA	비지도	선형	차원 축소	가장 기본적
LDA	지도	선형	분류 성능 개선	라벨 사용
t-SNE	비지도	비선형	시각화	군집 표현 우수
UMAP	비지도	비선형	시각화/축소	빠르고 대용량 적합
Autoencoder	비지도	비선형	압축/복원	딥러닝 기반
SVD	비지도	선형	행렬 압축	텍스트/추천
ICA	비지도	선형	독립 성분 분리	신호 처리
NMF	비지도	선형	토픽/패턴 추출	비음수 데이터
Isomap	비지도	비선형	곡면 구조 보존	매니폴드 학습

11. 어떤 알고리즘을 선택해야 하는가?

모델 학습 전 전처리 목적

PCA, SVD, LDA

데이터 시각화 목적

PCA, t-SNE, UMAP, Isomap

분류 성능 향상 목적

LDA, PCA

이미지나 음성 압축 목적

Autoencoder

텍스트 데이터 차원 축소

SVD, NMF

신호 분리

ICA

12. 실무 적용 기준

상황	추천 알고리즘
가장 기본적인 차원 축소	PCA
라벨이 있는 분류 문제	LDA
고차원 데이터를 2D로 시각화	t-SNE, UMAP
대용량 시각화	UMAP
텍스트 TF-IDF 축소	TruncatedSVD
이미지 압축	Autoencoder
음성·뇌파 신호 분리	ICA
토픽 추출	NMF

3. PCA

개념

PCA는 데이터 분산이 가장 큰 방향을 찾아 차원을 축소한다.
정답(label)을 사용하지 않는다.
비지도학습 기반 차원 축소이다.

계산 과정

Step 1. 평균 계산

데이터

$$X = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \end{bmatrix}$$

평균

$$\mu = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Step 2. 중심화(Centering)

평균 제거

$$X_c = X - \mu$$

결과

$$X_c = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}$$

Step 3. 공분산 행렬 계산

공식

$$C = \frac{1}{n-1} X_c^T X_c$$

결과

$$C = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

Step 4. 고유값 분해

공식

$$Cv = \lambda v$$

고유값

$$\begin{aligned} \lambda_1 &= 8 \\ \lambda_2 &= 0 \end{aligned}$$

고유벡터

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Step 5. 주성분 선택

가장 큰 고유값

$$\lambda_1$$

에 대응하는 고유벡터 선택

Step 6. 투영

공식

$$Z = X_c V$$

결과

원래

2차원

↓

1차원

으로 축소

설명 분산 비율

공식

$$\text{Explained Variance} = \frac{\lambda_i}{\sum \lambda}$$

예

$$\lambda = [8, 2, 1]$$

이면

$$PC1 = 72.7\%$$

$$PC2 = 18.2\%$$

$$PC3 = 9.1\%$$

이다.

PCA 특징

장점

- 계산 속도 빠름
- 노이즈 제거 가능
- 다중공선성 감소

단점

- 비선형 구조 반영 어려움
- 클래스 정보 사용 안 함

PCA 예제 코드

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pandas as pd

# 1. 데이터 로드
iris = load_iris()
X = iris.data
y = iris.target

# 2. 표준화
scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# 3. PCA 생성
pca = PCA(n_components=2)

# 4. PCA 적용
X_pca = pca.fit_transform(X_scaled)

# 5. 결과 확인
pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

pca_df["target"] = y

print(pca_df.head())

# 6. 설명 분산 비율 확인
print(pca.explained_variance_ratio_)
print("누적 설명 분산:", pca.explained_variance_ratio_.sum())

```

PyTorch 관점에서 차원 축소

PyTorch에서는 PCA를 직접 구현하기보다 보통 다음 방식을 사용한다.

1. sklearn PCA로 차원 축소 후 PyTorch 모델 입력
2. Autoencoder를 직접 만들어 잠재 벡터 사용

예:

```

원본 입력 100개
↓
PCA 적용
↓
20개 주성분
↓
PyTorch MLP 입력

```

이렇게 하면 학습 속도가 빨라지고 과적합이 줄어들 수 있다.

4. LDA

LDA는 Linear Discriminant Analysis, 즉 선형 판별 분석이다.
PCA와 달리 LDA는 정답 라벨을 사용한다.

PCA의 목적:

데이터가 가장 많이 퍼진 방향 찾기

LDA의 목적:

클래스가 가장 잘 구분되는 방향 찾기

LDA 계산 과정

예를 들어 두 개 클래스가 있다고 하자.

클래스 0:

$$D_0 = \{(2,3), (3,4)\}$$

클래스 1:

$$D_1 = \{(6,7), (7,8)\}$$

1단계. 클래스별 평균 계산

클래스 0 평균:

$$\mu_0 = \begin{bmatrix} \frac{2+3}{2} \\ \frac{3+4}{2} \end{bmatrix} = \begin{bmatrix} 2.5 \\ 3.5 \end{bmatrix}$$

클래스 1 평균:

$$\mu_1 = \begin{bmatrix} \frac{6+7}{2} \\ \frac{7+8}{2} \end{bmatrix} = \begin{bmatrix} 6.5 \\ 7.5 \end{bmatrix}$$

전체 평균:

$$\mu = \begin{bmatrix} 4.5 \\ 5.5 \end{bmatrix}$$

결과 해석:

클래스 0 중심: (2.5, 3.5)

클래스 1 중심: (6.5, 7.5)

전체 중심: (4.5, 5.5)

두 클래스의 중심이 떨어져 있으므로 분류 가능성이 높다.

2단계. 클래스 내 분산 계산

클래스 내 분산은 같은 클래스 안의 데이터들이 얼마나 흩어져 있는지를 의미한다.

$$S_w = \sum_{i=1}^c \sum_{x \in D_i} (x - \mu_i)(x - \mu_i)^T$$

클래스 0의 편차:

$$(2,3) - (2.5,3.5) = (-0.5, -0.5)$$

$$(3,4) - (2.5,3.5) = (0.5,0.5)$$

클래스 1의 편차:

$$(6,7) - (6.5,7.5) = (-0.5, -0.5)$$

$$(7,8) - (6.5,7.5) = (0.5,0.5)$$

각 편차의 외적은 다음과 같다.

$$\begin{bmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{bmatrix}$$

이 값이 총 4개 있으므로

$$S_w = 4 \begin{bmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

결과 해석:

S_w 가 작다

→ 같은 클래스 안의 데이터들이 서로 가깝다

3단계. 클래스 간 분산 계산

클래스 간 분산은 클래스 중심들이 전체 평균에서 얼마나 떨어져 있는지를 나타낸다.

$$S_b = \sum_{i=1}^c N_i (\mu_i - \mu)(\mu_i - \mu)^T$$

클래스 0:

$$\mu_0 - \mu = \begin{bmatrix} 2.5 - 4.5 \\ 3.5 - 5.5 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}$$

외적:

$$\begin{bmatrix} -2 \\ -2 \end{bmatrix} \begin{bmatrix} -2 & -2 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

클래스 0 샘플 수 $N_0 = 2$ 이므로

$$2 \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix} = \begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix}$$

클래스 1도 동일하게

$$\begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix}$$

따라서

$$S_b = \begin{bmatrix} 16 & 16 \\ 16 & 16 \end{bmatrix}$$

결과 해석:

S_b 가 크다

→ 클래스 중심들이 서로 멀리 떨어져 있다

4단계. 최적화 목표 계산

LDA는 다음 비율을 최대화한다.

$$J(W) = \frac{|W^T S_b W|}{|W^T S_w W|}$$

쉽게 말하면 다음을 원한다.

클래스 간 거리 크게

클래스 내부 거리 작게

즉,

S_b 는 크게

S_w 는 작게

만드는 축을 찾는다.

결과 해석

LDA 결과가 2차원 산점도에서 잘 분리된다면 다음과 같이 해석한다.

Low, Medium, High 클래스가 서로 다른 특성 패턴을 가진다.

반대로 색이 많이 겹치면 다음과 같이 해석한다.

입력 변수만으로 품질 등급을 명확히 나누기 어렵다.

5. t-SNE

t-SNE는 고차원 데이터의 가까운 이웃 관계를 2차원에서도 유지하려는 비선형 차원 축소 알고리즘이다.

PCA나 LDA는 직선 축을 찾지만, t-SNE는 복잡한 곡선 구조도 반영할 수 있다.

t-SNE 계산 과정

t-SNE는 거리 자체를 직접 보존하지 않고, 거리 관계를 확률로 바꾸어 비교한다.

1단계. 고차원 거리 계산

두 점 x_i, x_j 사이의 유클리드 거리를 계산한다.

$$d(x_i, x_j) = \|x_i - x_j\|$$

예를 들어

$$\begin{aligned}x_i &= (1, 2) \\x_j &= (4, 6)\end{aligned}$$

이면

$$\begin{aligned}d &= \sqrt{(1-4)^2 + (2-6)^2} \\&= \sqrt{9 + 16} \\&= 5\end{aligned}$$

결과 해석:

두 점 사이의 거리는 5

2단계. 거리 값을 유사도 확률로 변환

거리가 가까울수록 큰 확률을 갖게 만든다.

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

예를 들어 거리들이 다음과 같다고 하자.

A와 B 거리 = 1

A와 C 거리 = 5

그러면 t-SNE는 다음처럼 해석한다.

A는 C보다 B와 더 비슷하다.

즉,

$$p_{B|A} > p_{C|A}$$

가 된다.

3단계. 저차원에서도 유사도 계산

저차원 좌표 y_i, y_j 에서도 유사도를 계산한다.

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

여기서는 Student t-분포를 사용한다.

결과 해석:

q_{ij} 가 크다

→ 저차원에서 두 점이 가깝다

q_{ij} 가 작다

→ 저차원에서 두 점이 멀다

4단계. KL Divergence 최소화

고차원 유사도 p 와 저차원 유사도 q 가 비슷해지도록 학습한다.

$$KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

결과 해석:

KL 값이 크다

→ 고차원 구조와 저차원 구조가 많이 다름

KL 값이 작다

→ 고차원에서 가까운 점들이 저차원에서도 가깝게 배치됨

t-SNE는 이 KL 값을 줄이기 위해 점들의 2차원 위치를 반복적으로 조정한다.

t-SNE 결과값 해석

t-SNE 결과는 다음과 같은 2차원 좌표이다.

t-SNE 1	t-SNE 2
-12.5	3.1
8.2	-4.7
1.5	10.3

주의할 점은 다음과 같다.

t-SNE 1, t-SNE 2 자체의 값은 직접적인 의미가 약하다.

중요한 것은 다음이다.

같은 색 점들이 모여 있는가?

서로 다른 색 점들이 떨어져 있는가?

군집이 몇 개로 보이는가?

6. UMAP

UMAP은 Uniform Manifold Approximation and Projection의 약자이다.

t-SNE처럼 비선형 차원 축소 알고리즘이지만, 일반적으로 더 빠르고 대규모 데이터에 적합하다.

UMAP 계산 과정

UMAP은 데이터를 그래프로 보고, 고차원 그래프 구조를 저차원에서도 유지하려고 한다.

1단계. KNN 이웃 찾기

각 데이터에 대해 가까운 이웃 k 개를 찾는다.

예를 들어

`n_neighbors = 15`

이면 각 데이터마다 가장 가까운 15개 이웃을 찾는다.

결과 해석:

`n_neighbors` 작음

→ 가까운 지역 구조를 강조

`n_neighbors` 큼

→ 전체적인 데이터 구조를 더 반영

2단계. 고차원 그래프 가중치 계산

두 점 i, j 가 가까울수록 높은 연결 강도를 갖는다.

단순화하면 다음과 같이 볼 수 있다.

$$w_{ij} = e^{-d(i,j)}$$

예를 들어 거리 $d = 1$ 이면

$$w = e^{-1} = 0.3679$$

거리 $d = 5$ 이면

$$w = e^{-5} = 0.0067$$

결과 해석:

거리 1 → 연결 강도 0.3679

거리 5 → 연결 강도 0.0067

즉, 가까운 점은 강하게 연결되고 먼 점은 거의 연결되지 않는다.

3단계. 저차원 그래프 구성

저차원 좌표에서도 점들 사이의 연결 강도를 계산한다.

$$\hat{w}_{ij}$$

UMAP은 고차원 연결 강도 w_{ij} 와 저차원 연결 강도 $\hat{w}_{ij}$ 가 비슷해지도록 점들의 위치를 조정한다.

4단계. Cross Entropy 손실 최소화

UMAP은 다음과 같은 손실을 줄인다.

$$L = \sum_{i,j} \left[w_{ij} \log \frac{w_{ij}}{\hat{w}_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - \hat{w}_{ij}} \right]$$

결과 해석:

손실값이 크다

→ 고차원 이웃 구조와 저차원 이웃 구조가 다름

손실값이 작다

→ 고차원에서 가까운 데이터가 저차원에서도 가깝게 유지됨

UMAP 결과값 해석

UMAP 결과도 2차원 좌표이다.

UMAP 1 UMAP 2

3.2 8.1

3.5 7.9

-2.1 -4.3

이 숫자 자체보다 중요한 것은 점들의 배치이다.

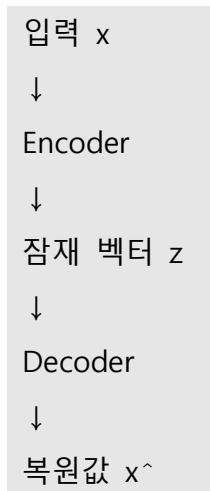
같은 품질 클래스가 모이면 좋은 결과

다른 품질 클래스와 많이 섞이면 구분이 어려운 결과

7. Autoencoder

Autoencoder는 딥러닝 기반 차원 축소 알고리즘이다.
입력 데이터를 압축한 뒤 다시 복원하도록 학습한다.

구조는 다음과 같다.



Autoencoder 계산 과정

예를 들어 입력 데이터가 11차원이라고 하자.

$$x = [x_1, x_2, \dots, x_{11}]$$

Autoencoder 구조는 다음과 같을 수 있다.

$$11 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 11$$

가운데 2차원 부분이 잠재 벡터이다.

1단계. Encoder 계산

Encoder는 입력을 압축한다.

$$z = f(Wx + b)$$

예를 들어 첫 번째 계층은 다음과 같다.

$$h_1 = \text{ReLU}(W_1x + b_1)$$

두 번째 계층:

$$h_2 = \text{ReLU}(W_2h_1 + b_2)$$

잠재 벡터:

$$z = W_3 h_2 + b_3$$

결과 해석:

원본 11개 변수

↓

신경망이 중요한 패턴만 압축

↓

2개 숫자 z_1, z_2

2단계. Decoder 계산

Decoder는 잠재 벡터를 다시 원본 차원으로 복원한다.

$$\hat{x} = g(W'z + b')$$

예를 들어

$$h_3 = \text{ReLU}(W_4 z + b_4)$$

$$h_4 = \text{ReLU}(W_5 h_3 + b_5)$$

$$\hat{x} = W_6 h_4 + b_6$$

결과 해석:

z_1, z_2 두 개 숫자

↓

원래 11개 변수 형태로 복원

3단계. 복원 오차 계산

원본 x 와 복원값 $\hat{x}$ 의 차이를 계산한다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

예를 들어 원본과 복원값이 다음과 같다고 하자.

원본 x = [1.0, 2.0, 3.0]

복원 $\hat{x}$ = [0.8, 2.2, 2.9]

차이:

[0.2, -0.2, 0.1]

제공:

[0.04, 0.04, 0.01]

평균:

$$MSE = \frac{0.04 + 0.04 + 0.01}{3} = 0.03$$

결과 해석:

MSE = 0.03

→ 복원 오차가 작음

→ Autoencoder가 원본 정보를 비교적 잘 압축함

4단계. 역전파와 가중치 업데이트

손실이 계산되면 역전파로 각 가중치의 수정 방향을 구한다.

$$\frac{\partial Loss}{\partial W}$$

그리고 Adam Optimizer가 가중치를 업데이트한다.

$$W = W - \eta \nabla L$$

여기서

η = learning rate

∇L = 손실 함수의 기울기

5단계. 반복 학습

학습은 여러 epoch 동안 반복된다.

Epoch 1 → Loss 0.85

Epoch 15 → Loss 0.62

Epoch 30 → Loss 0.48

Epoch 60 → Loss 0.35

Epoch 150 → Loss 0.22

결과 해석:

Loss가 감소한다

→ 복원 성능이 좋아지고 있다

Loss가 거의 줄지 않는다

→ 모델 구조나 학습률 조정 필요

Train Loss는 낮고 Test Loss는 높다

→ 과적합 가능성